# PDFExplainr: Contextual PDF Explainer

## Product Requirements Document & Development Report

**Date:** March 13, 2026
**Author:** Omkar Barve
**Status:** Ready for Development

---

## Executive Summary

**PDFExplainr** is a revolutionary PDF reading platform that solves the critical pain point for researchers reading dense technical documents: understanding complex terminology **without breaking reading flow**. Unlike existing solutions that force users into a chat interface, PDFExplainr enables natural reading with instant contextual explanations via text selection.

**Market Validation:** 8.5/10 viability score - highly useful, moderately unique, buildable in one week.

**Target Market:** 500,000+ ML researchers, quant finance students, medical students, and PhD candidates globally who regularly work with 500+ page technical documents.

---

## Problem Statement

### Current Pain Points

Researchers and students reading technical PDFs (machine learning papers, finance textbooks, medical journals) encounter specialized jargon every few paragraphs. Current solutions create severe workflow disruption:

**Traditional Approach:**

1. Encounter unfamiliar term ("deflated Sharpe ratio")
2. Open new tab or chat interface
3. Copy/paste or retype the term
4. Wait for response (3-10 seconds)
5. Read explanation
6. Return to PDF and relocate reading position
7. **Total time lost:** 30-60 seconds per query

**Impact:** Reading a 500-page technical book like "Machine Learning for Asset Management" by Marcos Lopez de Prado requires looking up 50-200 terms, resulting in **25-200 minutes of pure friction time** plus significant cognitive overhead from context switching.

### User Research

Based on analysis of existing tools (NotebookLM, PDF.ai, Humata, Mindgrasp):

- All force users to abandon reading and enter chat mode
- Average response time: 3-10 seconds
- Context precision: document-wide (less accurate than local context)
- Cognitive load: high (must formulate questions, switch interfaces)
- Reading flow: completely disrupted

---

## Solution Overview

### Core Innovation

**"Select-to-Explain" Contextual Reading:**

1. User reads PDF normally in full-page viewer
2. Encounters unfamiliar term or phrase
3. **Selects text** (standard highlight behavior)
4. **Sidebar appears instantly** (<1 second) with:
    - Simplified explanation in context
    - Source location within document
    - Related terms from the same document

5. User **continues reading** immediately - zero disruption

**Example Workflow:**

User reading Lopez de Prado's book selects "drawdown":

Sidebar displays:
"Drawdown: Maximum peak-to-trough decline in portfolio value. Lopez de Prado emphasizes drawdown control via metalearning strategies (Chapter 5, p.147). Related: Maximum Drawdown Duration, Calmar Ratio."

User continues reading in <2 seconds total.

## Key Differentiators

| Feature | PDFExplainr | Competitors |
|---|---|---|
| Reading Flow | Preserved | Disrupted |
| Response Time | <1 second | 3-10 seconds |
| Context Precision | ±50 tokens | Document-wide |
| TB File Support | Yes (chunked) | Limited/No |
| Offline Mode | Yes (local LLM) | Cloud-only |
| Workflow Steps | 2 (select → read) | 6+ (switch → ask → wait) |

Table 1: Competitive comparison matrix

---

# Target Users

## Primary Personas

**Persona 1: ML/Quant Researcher (Priority)**

- Reading: Technical finance/ML books (Lopez de Prado, Hastie, Bishop)
- Pain: Complex mathematical/domain terminology
- Volume: 200-500 pages/week
- Willingness to pay: High ($5-15/month)

**Persona 2: Medical Student**

- Reading: Medical textbooks, research papers
- Pain: Anatomical/pharmaceutical terminology
- Volume: 300-800 pages/week
- Willingness to pay: Medium ($3-10/month)

**Persona 3: PhD Candidate**

- Reading: Literature reviews, dissertations
- Pain: Interdisciplinary jargon
- Volume: 100-300 pages/week
- Willingness to pay: Medium-High ($5-12/month)

### Market Size

- Global PhD students: ~4.5 million
- Medical students: ~10 million
- ML/Data science professionals: ~15 million
- **Addressable market:** ~500,000 power users (1.7% penetration)
- **Target Year 1:** 5,000 users (1% penetration)

---

## Product Requirements

### MVP Features (Priority 0 - Week 1)

1. **TB-Scale PDF Viewer**
    - PDF.js integration with chunked loading
    - Support files up to 2GB (500+ pages)
    - Smooth scrolling and zoom
    - Mobile-responsive design
2. **Text Selection Handler**
    - Capture selected text on mouseup/touchend
    - Extract ±50 tokens of surrounding context
    - Handle multi-line selections
    - Preserve formatting context
3. **LLM Integration**
    - Local: Ollama + DeepSeek-Coder 7B
    - Cloud fallback: Groq API (free tier)
    - Prompt engineering for concise explanations
    - Response caching for repeated terms
4. **Sidebar Popup**

    - Non-blocking, dismissible interface
    - Loading states (<1 sec target)
    - Copy explanation to clipboard
    - Smooth animations (Tailwind)

### Priority 1 Features (Week 2-3)

1. **Persistent Annotations**
    - Save explanations to IndexedDB
    - Highlight previously explained terms
    - Export annotations to PDF notes
2. **Glossary Export**
    - CSV export of all saved terms
    - Anki flashcard generation
    - Markdown study guide
3. **Custom Prompts**
    - User-defined explanation style
    - Domain-specific templates (ML, Medicine, Law)
    - Explanation length control

### Priority 2 Features (Month 2-3)

- Multi-PDF cross-reference
- Team collaboration features
- Cloud sync across devices
- Browser extension version
- Integration with citation managers (Zotero, Mendeley)

---

## Technical Architecture

### Technology Stack

| Component | Technology | Rationale |
|---|---|---|
| Frontend | React 18 | Fast, component-based |
| PDF Rendering | PDF.js | TB support, Mozilla-backed |
| Styling | Tailwind CSS | Rapid prototyping |
| Icons | Lucide React | Lightweight, modern |
| LLM (Local) | Ollama + DeepSeek | Free, privacy-first |
| LLM (Cloud) | Groq API | 1M tokens/day free |
| Storage | IndexedDB | Offline annotations |
| Deployment | Vercel | Instant deploy, free tier |

Table 2: Technology stack rationale

## System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│ React Frontend (Vercel) │
├──────────────────────────────────────────────────────────────┤
│ PDF.js Viewer │ Selection Handler │
│ Sidebar UI │ Annotation Manager │
└──────────────┬──────────────────────────┬────────────────────┘
│ │
▼ ▼
┌──────────────┐ ┌──────────────────┐
│ Ollama │ │ IndexedDB │
│ (Local) │ │ (Storage) │
└──────────────┘ └──────────────────┘
│
▼ (fallback)
┌──────────────┐
│ Groq API │
│ (Cloud) │
└──────────────┘
```

## Data Flow

1. **PDF Loading:** Chunked loading via PDF.js (handles TB files)
2. **Selection Event:** User selects text → capture text + page coordinates
3. **Context Extraction:** PDF.js `getTextContent()` extracts ±50 tokens
4. **LLM Query:** POST to local Ollama or Groq API
5. **Response Display:** Sidebar renders explanation (<1s target)
6. **Persistence:** Save to IndexedDB for future reference

## LLM Prompt Engineering

**System Prompt:**
You are a contextual explainer for academic texts.
Explain selected terms simply and concisely.
Use surrounding context to provide precise, document-aware definitions.
Target audience: graduate students and researchers.
Keep explanations under 200 words, preferably 2-3 sentences.

**Query Template:**
Explain the term '{selected_text}' in this context:

Previous context: {prev_50_tokens}
SELECTED: {selected_text}
Following context: {next_50_tokens}

Document domain: {detected_domain} (e.g., ML/Finance, Medicine, Law)

Provide:

1. Simple explanation
2. Relevance in this document context
3. One related term if applicable

---

# Competitive Analysis

## Direct Competitors

### Google NotebookLM

- **Strengths:** Free, 500K word capacity, AI podcast generation
- **Weaknesses:** Chat-only interface, no in-flow explanations
- **Our advantage:** 5x faster workflow, reading flow preservation

### PDF.ai

- **Strengths:** Good UI, popular
- **Weaknesses:** 100MB file limit, chat-only, $10/month
- **Our advantage:** TB support, select-to-explain, offline mode

### Humata AI

- **Strengths:** Technical accuracy, STEM-focused
- **Weaknesses:** Chat interface, expensive at scale
- **Our advantage:** Zero context-switch, faster

### ReaderBro

- **Strengths:** Select-to-chat functionality
- **Weaknesses:** Unknown file limits, paid-only, limited context
- **Our advantage:** Better context precision, open source

### AI Dictionary Extension

- **Strengths:** Webpage popup, free
- **Weaknesses:** No PDF support, generic definitions
- **Our advantage:** Document-aware, TB PDFs, technical domains

## Unique Value Proposition

**"The only PDF reader that explains without interrupting."**

Nobody else combines:

1. Natural reading flow (no chat interface)
2. TB-scale file support
3. Document-aware context (±50 tokens)

4. <1 second response time
5. Offline-first capability

---

# Development Roadmap

## Week 1: MVP Core

### Day 1-2: Foundation

- React app scaffolding
- PDF.js integration
- Basic text selection
- Mock sidebar UI

### Day 3-4: LLM Integration

- Ollama local setup
- Context extraction logic
- Prompt engineering
- API endpoint creation

### Day 5-6: Polish

- Loading states and animations
- Dark mode support
- Error handling
- Basic persistence

### Day 7: Deploy

- Vercel deployment
- Landing page
- GitHub repository
- Demo video

## Week 2-3: Enhancement

- Persistent annotations system
- Export functionality (CSV, Anki)
- Custom prompts UI
- Performance optimization

- User testing with 50 beta users

### Month 2-3: Growth Features

- Multi-PDF workspace
- Cloud sync (optional)
- Browser extension
- Mobile app (React Native)
- Integration partnerships

---

## Success Metrics

### North Star Metric

**Explanations per reading session** - measures core value delivery

### Key Performance Indicators

**Week 1 (MVP Launch)**

- 100 users (GitHub, HackerNews)
- 50 active testers
- 10+ GitHub stars
- <2s average response time

**Month 1**

- 1,000 users
- 500 GitHub stars
- 100 daily active users
- 50 paid conversions (Pro tier)

**Month 3**

- 5,000 total users
- 2,000 GitHub stars
- 500 daily active users
- $2,000 MRR

**Year 1**

- 50,000 total users
- 10,000 GitHub stars
- 5,000 daily active users
- $10,000 MRR

---

# Monetization Strategy

## Pricing Tiers

### Free Tier

- 3 PDFs per day
- Local LLM only (Ollama)
- Basic export (CSV)
- Community support

### Pro Tier ($5/month)

- Unlimited PDFs
- Cloud LLM (faster, better quality)
- Advanced exports (Anki, Markdown)
- Custom prompts
- Priority support

### Academic Tier ($15/year - 75% discount)

- All Pro features
- Team collaboration (up to 5)
- Citation manager integration
- Dedicated academic support

### Enterprise (Custom)

- Self-hosted deployment
- Custom LLM fine-tuning
- SSO integration
- SLA guarantees

### Revenue Projections

| Timeline | Users | Conversion | MRR |
|---|---|---|---|
| Month 1 | 1,000 | 5% | $250 |
| Month 3 | 5,000 | 8% | $2,000 |
| Month 6 | 15,000 | 10% | $7,500 |
| Year 1 | 50,000 | 12% | $30,000 |

Table 3: Revenue projections (conservative)

---

## Go-to-Market Strategy

### Phase 1: Developer Community (Week 1)

1. **GitHub Repository**
   - Open source core (MIT license)
   - Comprehensive README with demo GIF
   - Clear contribution guidelines
2. **HackerNews Launch**
   - "Show HN: PDFExplainr - Select any term in TB PDFs for instant explanations"
   - Demo with Lopez de Prado book
   - Technical implementation details
3. **ProductHunt**
   - Launch on Tuesday/Wednesday (optimal)
   - Maker story: solving personal pain point
   - Video demo showing speed advantage

### Phase 2: Academic Communities (Week 2-4)

- Reddit: r/MachineLearning, r/quant, r/medicalschool, r/PhD
- Twitter: Tag ML researchers, Lopez de Prado followers
- Academic forums: ResearchGate, Academia.edu
- University partnerships: MIT, Stanford ML reading groups

### Phase 3: Content Marketing (Month 2-3)

- Blog: "How I read 500-page ML textbooks 5x faster"
- YouTube: Demo videos, comparison with NotebookLM
- Case studies: "ML researchers using PDFExplainr"
- Podcast interviews: AI tools, productivity

### Phase 4: Partnerships (Month 4-6)

- Zotero/Mendeley integration
- University library systems
- Academic publishers (Springer, Elsevier)
- Online course platforms (Coursera, edX)

---

# Risk Assessment & Mitigation

## Technical Risks

#### Risk 1: PDF.js Text Extraction Accuracy

- **Impact:** High - Core functionality breaks
- **Probability:** Medium (complex layouts, scanned PDFs)
- **Mitigation:**
  - Fallback to OCR (Tesseract.js) for scanned PDFs
  - User feedback loop for extraction errors
  - Whitelist known-good publishers

#### Risk 2: Local LLM Speed

- **Impact:** Medium - Response time >2s breaks UX
- **Probability:** Medium (depends on user hardware)
- **Mitigation:**
  - Cloud LLM fallback (Groq free tier)
  - Response caching for repeated terms
  - Progressive enhancement (show partial results)

#### Risk 3: TB File Performance

- **Impact:** Medium - Browser crashes on large files
- **Probability:** Low (PDF.js designed for this)
- **Mitigation:**

    - Aggressive chunking (load 10 pages at a time)
    - Virtual scrolling
    - Memory management optimization

## Business Risks

**Risk 4: Low User Adoption**

- **Impact:** High - Product fails to gain traction
- **Probability:** Medium (crowded market)
- **Mitigation:**
    - Solve specific pain point better than anyone
    - Open source = network effects
    - Target niche first (ML/quant researchers)

**Risk 5: Competitor Copy**

- **Impact:** Medium - NotebookLM/PDF.ai add feature
- **Probability:** Low-Medium (6-12 month lag)
- **Mitigation:**
    - First-mover advantage
    - Community building
    - Faster iteration cycle
    - Open source moat

**Risk 6: LLM API Costs**

- **Impact:** High - Unit economics break
- **Probability:** Low (local-first design)
- **Mitigation:**
    - Default to local LLM
    - Response caching (90% hit rate expected)
    - Freemium limits
    - Groq free tier (1M tokens/day)

## Regulatory Risks

**Risk 7: Copyright/Fair Use**

- **Impact:** Medium - Legal challenges
- **Probability:** Low (tool, not content repository)
- **Mitigation:**

    - Users upload their own PDFs
    - No content hosting
    - Clear ToS around fair use

---

# Validation Checklist

## Problem Validation

- ✅ Personal pain point (reading Lopez de Prado)
- ✅ Observed in academic communities (Reddit complaints)
- ✅ Existing solutions inadequate (all force chat interface)
- ✅ Willingness to pay (similar tools charge $5-20/month)

## Solution Validation

- ✅ Technical feasibility (PDF.js + Ollama proven)
- ✅ Performance targets achievable (<1s response)
- ✅ Differentiation clear (reading flow preservation)
- ✅ MVP buildable in 1 week

## Market Validation

- ✅ 500K+ addressable users
- ✅ Existing paid alternatives prove market
- ✅ Distribution channels identified (GitHub, HN, Reddit)
- ✅ Monetization path clear (freemium SaaS)

---

# Next Steps

## Immediate Actions (This Week)

1. **Set up development environment**
    - Initialize React app
    - Install dependencies (PDF.js, Tailwind)
    - Configure Ollama locally
2. **Build MVP core**
    - PDF viewer with text selection
    - LLM integration
    - Basic sidebar UI
3. **Deploy alpha version**

    - Vercel deployment
    - Test with Lopez de Prado PDF
    - Gather initial feedback

### Short-term (Month 1)

1. Launch on HackerNews and ProductHunt
2. Iterate based on user feedback
3. Add persistent annotations
4. Reach 1,000 users

### Medium-term (Months 2-3)

1. Implement Pro tier
2. Build export functionality
3. Create content marketing assets
4. Reach $2,000 MRR

### Long-term (Year 1)

1. Browser extension
2. Mobile apps
3. Partnership integrations
4. Scale to 50,000 users, $30,000 MRR

---

## Conclusion

PDFExplainr addresses a genuine, underserved need in the academic and research community. By preserving natural reading flow while providing instant contextual explanations, it offers a superior user experience to existing chat-based PDF tools.

**Key Success Factors:**

1. **Speed:** <1 second response time vs 3-10 seconds for competitors
2. **Flow:** Zero context-switching vs complete workflow disruption
3. **Scale:** TB file support vs 100MB limits
4. **Precision:** ±50 token context vs document-wide generic answers

**Investment Required:** Minimal - MVP buildable in 1 week with zero capital

**Risk Level:** Low-Medium - validated problem, proven technology
**Upside Potential:** High - 500K addressable users, clear monetization path

**Recommendation:** BUILD IMMEDIATELY. This is a thesis side-project that could become a 10K-star GitHub repository and sustainable SaaS business.

The market is ready. The technology is proven. The differentiation is clear.

**Time to ship.**

---

## Appendix A: Technology Setup

### Prerequisites

- Node.js 18+ installed
- Ollama installed (for local LLM)
- Git and GitHub account
- Vercel account (free tier)

### Initial Setup Commands

# Create React app

npx create-react-app pdfexplainr
cd pdfexplainr

# Install dependencies

npm install pdfjs-dist @tailwindcss/forms lucide-react
npm install -D tailwindcss postcss autoprefixer

# Initialize Tailwind

npx tailwindcss init -p

# Install Ollama (Mac/Linux)

curl https://ollama.ai/install.sh | sh

# Pull DeepSeek model

ollama pull deepseek-coder:7b

# Start development

npm start

### Environment Variables

REACT_APP_OLLAMA_URL=http://localhost:11434
REACT_APP_GROQ_API_KEY=your_groq_key_here
REACT_APP_MAX_PDF_SIZE_MB=2048

---

## Appendix B: LLM Comparison

| Model | Speed | Quality | Cost | Offline |
|---|---|---|---|---|
| DeepSeek-Coder 7B | Fast | Good | Free | Yes |
| Llama 3.2 8B | Medium | Better | Free | Yes |
| Groq (Cloud) | Fastest | Best | Free tier | No |
| GPT-4o-mini | Fast | Best | $0.15/1M | No |

Table 4: LLM options for MVP

**Recommendation:** DeepSeek-Coder 7B for MVP (best balance of speed/quality/cost).

---

## Appendix C: Competitive Feature Matrix

| Feature | PDFExplainr | NotebookLM | PDF.ai | Humata | ReaderI |
|---|---|---|---|---|---|
| Select-to-Explain | ✓ | ✗ | ✗ | ✗ | ✓ |
| TB Files | ✓ | ✓ | ✗ | ✓ | ? |
| <1s Response | ✓ | ✗ | ✗ | ✗ | ✗ |
| Offline Mode | ✓ | ✗ | ✗ | ✗ | ✗ |
| Context Precision | ✓ | ✗ | ✗ | ~ | ✗ |
| Free Tier | ✓ | ✓ | ✗ | ✓ | ✗ |
| Open Source | ✓ | ✗ | ✗ | ✗ | ✗ |

Table 5: Full competitive analysis (✓ = yes, ✗ = no, ~ = partial, ? = unknown)

---

*End of Report*

# PDFExplainr: Contextual PDF Explainer

## Product Requirements Document & Development Report

**Date:** March 13, 2026
**Author:** Omkar Barve
**Status:** Ready for Development

---

# Executive Summary

**PDFExplainr** is a revolutionary PDF reading platform that solves the critical pain point for researchers reading dense technical documents: understanding complex terminology **without breaking reading flow**. Unlike existing solutions that force users into a chat interface, PDFExplainr enables natural reading with instant contextual explanations via text selection.

**Market Validation:** 8.5/10 viability score - highly useful, moderately unique, buildable in one week.

**Target Market:** 500,000+ ML researchers, quant finance students, medical students, and PhD candidates globally who regularly work with 500+ page technical documents.

---

# Problem Statement

## Current Pain Points

Researchers and students reading technical PDFs (machine learning papers, finance textbooks, medical journals) encounter specialized jargon every few paragraphs. Current solutions create severe workflow disruption:

**Traditional Approach:**

1. Encounter unfamiliar term ("deflated Sharpe ratio")
2. Open new tab or chat interface
3. Copy/paste or retype the term
4. Wait for response (3-10 seconds)
5. Read explanation
6. Return to PDF and relocate reading position
7. **Total time lost:** 30-60 seconds per query

**Impact:** Reading a 500-page technical book like "Machine Learning for Asset Management" by Marcos Lopez de Prado requires looking up 50-200 terms, resulting in **25-200 minutes of pure friction time** plus significant cognitive overhead from context switching.

### User Research

Based on analysis of existing tools (NotebookLM, PDF.ai, Humata, Mindgrasp):

- All force users to abandon reading and enter chat mode
- Average response time: 3-10 seconds
- Context precision: document-wide (less accurate than local context)
- Cognitive load: high (must formulate questions, switch interfaces)
- Reading flow: completely disrupted

---

## Solution Overview

### Core Innovation

**"Select-to-Explain" Contextual Reading:**

1. User reads PDF normally in full-page viewer
2. Encounters unfamiliar term or phrase
3. **Selects text** (standard highlight behavior)
4. **Sidebar appears instantly** (<1 second) with:
    - Simplified explanation in context
    - Source location within document
    - Related terms from the same document
5. User **continues reading** immediately - zero disruption

**Example Workflow:**

User reading Lopez de Prado's book selects "drawdown":

Sidebar displays:
"Drawdown: Maximum peak-to-trough decline in portfolio value. Lopez de Prado emphasizes drawdown control via metalearning strategies (Chapter 5, p.147). Related: Maximum Drawdown Duration, Calmar Ratio."

User continues reading in <2 seconds total.

### Key Differentiators

| Feature | PDFExplainr | Competitors |
|---|---|---|
| Reading Flow | Preserved | Disrupted |
| Response Time | <1 second | 3-10 seconds |
| Context Precision | ±50 tokens | Document-wide |
| TB File Support | Yes (chunked) | Limited/No |
| Offline Mode | Yes (local LLM) | Cloud-only |
| Workflow Steps | 2 (select → read) | 6+ (switch → ask → wait) |

Table 6: Competitive comparison matrix

---

# Target Users

## Primary Personas

**Persona 1: ML/Quant Researcher (Priority)**

- Reading: Technical finance/ML books (Lopez de Prado, Hastie, Bishop)
- Pain: Complex mathematical/domain terminology
- Volume: 200-500 pages/week
- Willingness to pay: High ($5-15/month)

**Persona 2: Medical Student**

- Reading: Medical textbooks, research papers
- Pain: Anatomical/pharmaceutical terminology
- Volume: 300-800 pages/week
- Willingness to pay: Medium ($3-10/month)

**Persona 3: PhD Candidate**

- Reading: Literature reviews, dissertations
- Pain: Interdisciplinary jargon
- Volume: 100-300 pages/week
- Willingness to pay: Medium-High ($5-12/month)

### Market Size

- Global PhD students: ~4.5 million
- Medical students: ~10 million
- ML/Data science professionals: ~15 million
- **Addressable market:** ~500,000 power users (1.7% penetration)
- **Target Year 1:** 5,000 users (1% penetration)

---

# Product Requirements

## MVP Features (Priority 0 - Week 1)

1. **TB-Scale PDF Viewer**
    - PDF.js integration with chunked loading
    - Support files up to 2GB (500+ pages)
    - Smooth scrolling and zoom
    - Mobile-responsive design
2. **Text Selection Handler**
    - Capture selected text on mouseup/touchend
    - Extract ±50 tokens of surrounding context
    - Handle multi-line selections
    - Preserve formatting context
3. **LLM Integration**
    - Local: Ollama + DeepSeek-Coder 7B
    - Cloud fallback: Groq API (free tier)
    - Prompt engineering for concise explanations
    - Response caching for repeated terms
4. **Sidebar Popup**
    - Non-blocking, dismissible interface
    - Loading states (<1 sec target)
    - Copy explanation to clipboard
    - Smooth animations (Tailwind)

## Priority 1 Features (Week 2-3)

1. **Persistent Annotations**
    - Save explanations to IndexedDB
    - Highlight previously explained terms
    - Export annotations to PDF notes

2. **Glossary Export**
    - CSV export of all saved terms
    - Anki flashcard generation
    - Markdown study guide
3. **Custom Prompts**
    - User-defined explanation style
    - Domain-specific templates (ML, Medicine, Law)
    - Explanation length control

### Priority 2 Features (Month 2-3)

- Multi-PDF cross-reference
- Team collaboration features
- Cloud sync across devices
- Browser extension version
- Integration with citation managers (Zotero, Mendeley)

---

## Technical Architecture

### Technology Stack

| Component | Technology | Rationale |
|---|---|---|
| Frontend | React 18 | Fast, component-based |
| PDF Rendering | PDF.js | TB support, Mozilla-backed |
| Styling | Tailwind CSS | Rapid prototyping |
| Icons | Lucide React | Lightweight, modern |
| LLM (Local) | Ollama + DeepSeek | Free, privacy-first |
| LLM (Cloud) | Groq API | 1M tokens/day free |
| Storage | IndexedDB | Offline annotations |
| Deployment | Vercel | Instant deploy, free tier |

Table 7: Technology stack rationale

## System Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│ React Frontend (Vercel) │
├──────────────────────────────────────────────────────────────────────────────┤
│ PDF.js Viewer │ Selection Handler │
│ Sidebar UI │ Annotation Manager │
└──────────────┬──────────────────────────────┬──────────────────────────────┘
│ │
▼ ▼
┌──────────────┐ ┌──────────────────┐
│ Ollama │ │ IndexedDB │
│ (Local) │ │ (Storage) │
└──────────────┘ └──────────────────┘
│
▼ (fallback)
┌──────────────┐
│ Groq API │
│ (Cloud) │
└──────────────┘
```

## Data Flow

1. **PDF Loading:** Chunked loading via PDF.js (handles TB files)
2. **Selection Event:** User selects text → capture text + page coordinates
3. **Context Extraction:** PDF.js `getTextContent()` extracts ±50 tokens
4. **LLM Query:** POST to local Ollama or Groq API
5. **Response Display:** Sidebar renders explanation (<1s target)
6. **Persistence:** Save to IndexedDB for future reference

## LLM Prompt Engineering

**System Prompt:**
You are a contextual explainer for academic texts.
Explain selected terms simply and concisely.
Use surrounding context to provide precise, document-aware definitions.

Target audience: graduate students and researchers.
Keep explanations under 200 words, preferably 2-3 sentences.

**Query Template:**
Explain the term '{selected_text}' in this context:

Previous context: {prev_50_tokens}
SELECTED: {selected_text}
Following context: {next_50_tokens}

Document domain: {detected_domain} (e.g., ML/Finance, Medicine, Law)

Provide:

1. Simple explanation
2. Relevance in this document context
3. One related term if applicable

---

# Competitive Analysis

## Direct Competitors

### Google NotebookLM

- **Strengths:** Free, 500K word capacity, AI podcast generation
- **Weaknesses:** Chat-only interface, no in-flow explanations
- **Our advantage:** 5x faster workflow, reading flow preservation

### PDF.ai

- **Strengths:** Good UI, popular
- **Weaknesses:** 100MB file limit, chat-only, $10/month
- **Our advantage:** TB support, select-to-explain, offline mode

### Humata AI

- **Strengths:** Technical accuracy, STEM-focused
- **Weaknesses:** Chat interface, expensive at scale
- **Our advantage:** Zero context-switch, faster

### ReaderBro

- **Strengths:** Select-to-chat functionality
- **Weaknesses:** Unknown file limits, paid-only, limited context
- **Our advantage:** Better context precision, open source

**AI Dictionary Extension**

- **Strengths:** Webpage popup, free
- **Weaknesses:** No PDF support, generic definitions
- **Our advantage:** Document-aware, TB PDFs, technical domains

### Unique Value Proposition

**"The only PDF reader that explains without interrupting."**

Nobody else combines:

1. Natural reading flow (no chat interface)
2. TB-scale file support
3. Document-aware context (±50 tokens)
4. <1 second response time
5. Offline-first capability

---

# Development Roadmap

## Week 1: MVP Core

**Day 1-2: Foundation**

- React app scaffolding
- PDF.js integration
- Basic text selection
- Mock sidebar UI

**Day 3-4: LLM Integration**

- Ollama local setup
- Context extraction logic
- Prompt engineering
- API endpoint creation

**Day 5-6: Polish**

- Loading states and animations
- Dark mode support
- Error handling
- Basic persistence

**Day 7: Deploy**

- Vercel deployment
- Landing page
- GitHub repository
- Demo video

### Week 2-3: Enhancement

- Persistent annotations system
- Export functionality (CSV, Anki)
- Custom prompts UI
- Performance optimization
- User testing with 50 beta users

### Month 2-3: Growth Features

- Multi-PDF workspace
- Cloud sync (optional)
- Browser extension
- Mobile app (React Native)
- Integration partnerships

---

## Success Metrics

### North Star Metric

**Explanations per reading session** - measures core value delivery

### Key Performance Indicators

**Week 1 (MVP Launch)**

- 100 users (GitHub, HackerNews)
- 50 active testers
- 10+ GitHub stars

- <2s average response time

**Month 1**

- 1,000 users
- 500 GitHub stars
- 100 daily active users
- 50 paid conversions (Pro tier)

**Month 3**

- 5,000 total users
- 2,000 GitHub stars
- 500 daily active users
- $2,000 MRR

**Year 1**

- 50,000 total users
- 10,000 GitHub stars
- 5,000 daily active users
- $10,000 MRR

---

# Monetization Strategy

## Pricing Tiers

**Free Tier**

- 3 PDFs per day
- Local LLM only (Ollama)
- Basic export (CSV)
- Community support

**Pro Tier ($5/month)**

- Unlimited PDFs
- Cloud LLM (faster, better quality)
- Advanced exports (Anki, Markdown)
- Custom prompts
- Priority support

**Academic Tier ($15/year - 75% discount)**

- All Pro features
- Team collaboration (up to 5)
- Citation manager integration
- Dedicated academic support

**Enterprise (Custom)**

- Self-hosted deployment
- Custom LLM fine-tuning
- SSO integration
- SLA guarantees

### Revenue Projections

| Timeline | Users | Conversion | MRR |
|---|---|---|---|
| Month 1 | 1,000 | 5% | $250 |
| Month 3 | 5,000 | 8% | $2,000 |
| Month 6 | 15,000 | 10% | $7,500 |
| Year 1 | 50,000 | 12% | $30,000 |

Table 8: Revenue projections (conservative)

---

## Go-to-Market Strategy

### Phase 1: Developer Community (Week 1)

1. **GitHub Repository**
    - Open source core (MIT license)
    - Comprehensive README with demo GIF
    - Clear contribution guidelines
2. **HackerNews Launch**
    - "Show HN: PDFExplainr - Select any term in TB PDFs for instant explanations"
    - Demo with Lopez de Prado book
    - Technical implementation details
3. **ProductHunt**
    - Launch on Tuesday/Wednesday (optimal)

- Maker story: solving personal pain point
- Video demo showing speed advantage

### Phase 2: Academic Communities (Week 2-4)

- Reddit: r/MachineLearning, r/quant, r/medicalschool, r/PhD
- Twitter: Tag ML researchers, Lopez de Prado followers
- Academic forums: ResearchGate, Academia.edu
- University partnerships: MIT, Stanford ML reading groups

### Phase 3: Content Marketing (Month 2-3)

- Blog: "How I read 500-page ML textbooks 5x faster"
- YouTube: Demo videos, comparison with NotebookLM
- Case studies: "ML researchers using PDFExplainr"
- Podcast interviews: AI tools, productivity

### Phase 4: Partnerships (Month 4-6)

- Zotero/Mendeley integration
- University library systems
- Academic publishers (Springer, Elsevier)
- Online course platforms (Coursera, edX)

---

# Risk Assessment & Mitigation

## Technical Risks

#### Risk 1: PDF.js Text Extraction Accuracy

- **Impact:** High - Core functionality breaks
- **Probability:** Medium (complex layouts, scanned PDFs)
- **Mitigation:**
  - Fallback to OCR (Tesseract.js) for scanned PDFs
  - User feedback loop for extraction errors
  - Whitelist known-good publishers

#### Risk 2: Local LLM Speed

- **Impact:** Medium - Response time >2s breaks UX
- **Probability:** Medium (depends on user hardware)

- **Mitigation:**
  - Cloud LLM fallback (Groq free tier)
  - Response caching for repeated terms
  - Progressive enhancement (show partial results)

**Risk 3: TB File Performance**

- **Impact:** Medium - Browser crashes on large files
- **Probability:** Low (PDF.js designed for this)
- **Mitigation:**
  - Aggressive chunking (load 10 pages at a time)
  - Virtual scrolling
  - Memory management optimization

## Business Risks

**Risk 4: Low User Adoption**

- **Impact:** High - Product fails to gain traction
- **Probability:** Medium (crowded market)
- **Mitigation:**
  - Solve specific pain point better than anyone
  - Open source = network effects
  - Target niche first (ML/quant researchers)

**Risk 5: Competitor Copy**

- **Impact:** Medium - NotebookLM/PDF.ai add feature
- **Probability:** Low-Medium (6-12 month lag)
- **Mitigation:**
  - First-mover advantage
  - Community building
  - Faster iteration cycle
  - Open source moat

**Risk 6: LLM API Costs**

- **Impact:** High - Unit economics break
- **Probability:** Low (local-first design)
- **Mitigation:**
  - Default to local LLM
  - Response caching (90% hit rate expected)

    - Freemium limits
    - Groq free tier (1M tokens/day)

### Regulatory Risks

**Risk 7: Copyright/Fair Use**

- **Impact:** Medium - Legal challenges
- **Probability:** Low (tool, not content repository)
- **Mitigation:**
    - Users upload their own PDFs
    - No content hosting
    - Clear ToS around fair use

---

## Validation Checklist

### Problem Validation

- ✅ Personal pain point (reading Lopez de Prado)
- ✅ Observed in academic communities (Reddit complaints)
- ✅ Existing solutions inadequate (all force chat interface)
- ✅ Willingness to pay (similar tools charge $5-20/month)

### Solution Validation

- ✅ Technical feasibility (PDF.js + Ollama proven)
- ✅ Performance targets achievable (<1s response)
- ✅ Differentiation clear (reading flow preservation)
- ✅ MVP buildable in 1 week

### Market Validation

- ✅ 500K+ addressable users
- ✅ Existing paid alternatives prove market
- ✅ Distribution channels identified (GitHub, HN, Reddit)
- ✅ Monetization path clear (freemium SaaS)

---

# Next Steps

## Immediate Actions (This Week)

1. **Set up development environment**
    - Initialize React app
    - Install dependencies (PDF.js, Tailwind)
    - Configure Ollama locally
2. **Build MVP core**
    - PDF viewer with text selection
    - LLM integration
    - Basic sidebar UI
3. **Deploy alpha version**
    - Vercel deployment
    - Test with Lopez de Prado PDF
    - Gather initial feedback

## Short-term (Month 1)

1. Launch on HackerNews and ProductHunt
2. Iterate based on user feedback
3. Add persistent annotations
4. Reach 1,000 users

## Medium-term (Months 2-3)

1. Implement Pro tier
2. Build export functionality
3. Create content marketing assets
4. Reach $2,000 MRR

## Long-term (Year 1)

1. Browser extension
2. Mobile apps
3. Partnership integrations
4. Scale to 50,000 users, $30,000 MRR

---

# Conclusion

PDFExplainr addresses a genuine, underserved need in the academic and research community. By preserving natural reading flow while providing instant contextual explanations, it offers a superior user experience to existing chat-based PDF tools.

**Key Success Factors:**

1. **Speed:** <1 second response time vs 3-10 seconds for competitors
2. **Flow:** Zero context-switching vs complete workflow disruption
3. **Scale:** TB file support vs 100MB limits
4. **Precision:** ±50 token context vs document-wide generic answers

**Investment Required:** Minimal - MVP buildable in 1 week with zero capital
**Risk Level:** Low-Medium - validated problem, proven technology
**Upside Potential:** High - 500K addressable users, clear monetization path

**Recommendation:** BUILD IMMEDIATELY. This is a thesis side-project that could become a 10K-star GitHub repository and sustainable SaaS business.

The market is ready. The technology is proven. The differentiation is clear.

**Time to ship.**

---

# Appendix A: Technology Setup

## Prerequisites

- Node.js 18+ installed
- Ollama installed (for local LLM)
- Git and GitHub account
- Vercel account (free tier)

### Initial Setup Commands

# Create React app

npx create-react-app pdfexplainr
cd pdfexplainr

# Install dependencies

npm install pdfjs-dist @tailwindcss/forms lucide-react
npm install -D tailwindcss postcss autoprefixer

# Initialize Tailwind

npx tailwindcss init -p

# Install Ollama (Mac/Linux)

curl https://ollama.ai/install.sh | sh

# Pull DeepSeek model

ollama pull deepseek-coder:7b

# Start development

npm start

### Environment Variables

REACT_APP_OLLAMA_URL=http://localhost:11434
REACT_APP_GROQ_API_KEY=your_groq_key_here
REACT_APP_MAX_PDF_SIZE_MB=2048

---

# Appendix B: LLM Comparison

| Model | Speed | Quality | Cost | Offline |
|---|---|---|---|---|
| DeepSeek-Coder 7B | Fast | Good | Free | Yes |
| Llama 3.2 8B | Medium | Better | Free | Yes |
| Groq (Cloud) | Fastest | Best | Free tier | No |
| GPT-4o-mini | Fast | Best | $0.15/1M | No |

Table 9: LLM options for MVP

**Recommendation:** DeepSeek-Coder 7B for MVP (best balance of speed/quality/cost).

---

# Appendix C: Competitive Feature Matrix

| Feature | PDFExplainr | NotebookLM | PDF.ai | Humata | ReaderI |
|---|---|---|---|---|---|
| Select-to-Explain | ✓ | ✗ | ✗ | ✗ | ✓ |
| TB Files | ✓ | ✓ | ✗ | ✓ | ? |
| <1s Response | ✓ | ✗ | ✗ | ✗ | ✗ |
| Offline Mode | ✓ | ✗ | ✗ | ✗ | ✗ |
| Context Precision | ✓ | ✗ | ✗ | ~ | ✗ |
| Free Tier | ✓ | ✓ | ✗ | ✓ | ✗ |
| Open Source | ✓ | ✗ | ✗ | ✗ | ✗ |

Table 10: Full competitive analysis (✓ = yes, ✗ = no, ~ = partial, ? = unknown)

---

*End of Report*